



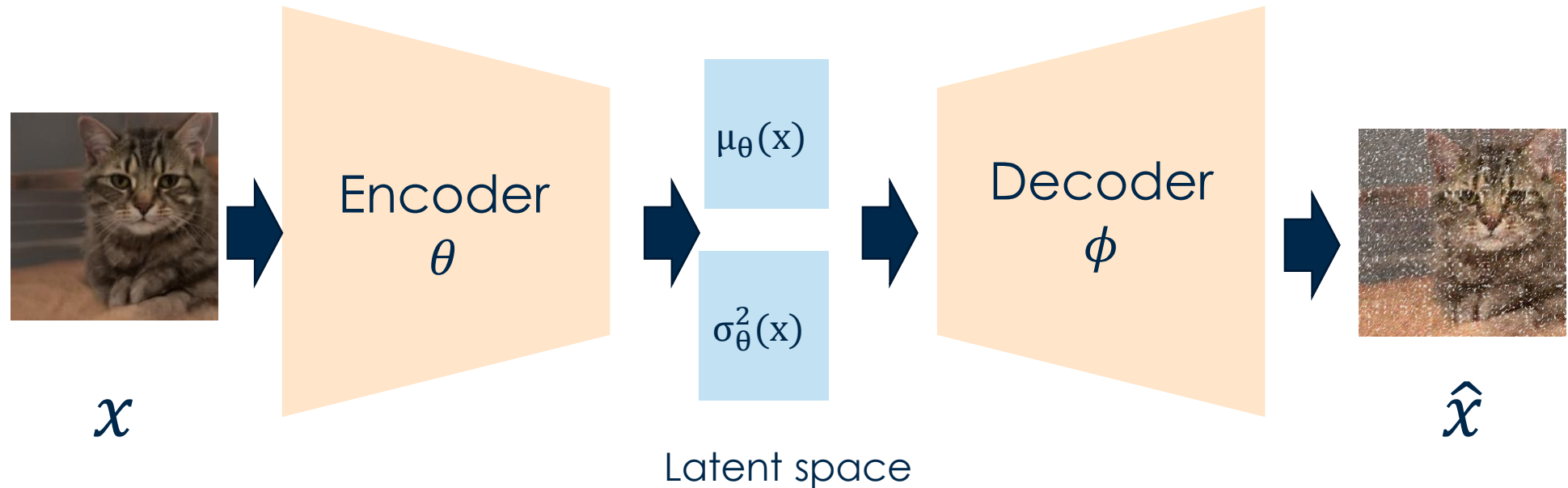
Politecnico
di Torino

Generative Artificial Intelligence for graphics and multimedia

Normalizing flows
Prof. Lia Morra



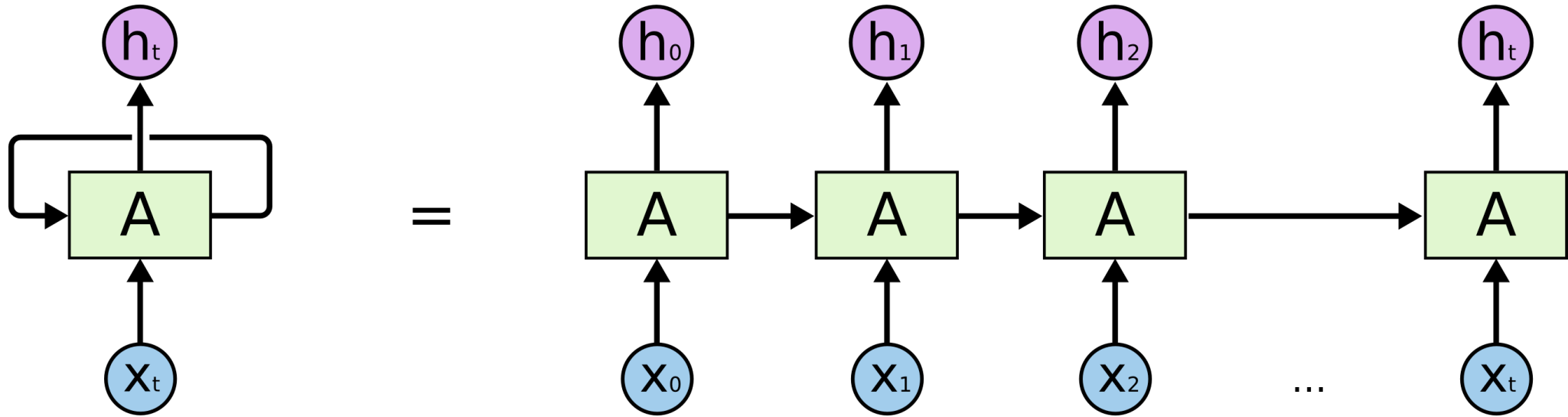
Recap: Variational autencoder



Variational autoencoders can learn feature representations (via latent variables $\mathbf{z}$) but the training objective is intractable

$$p_{\phi}(x) = \int p_{\phi}(x|z)p(z)dz$$

Recap: autoregressive models



Autoregressive models provide tractable likelihoods but no direct mechanism for learning features

$$p(\mathbf{x}) = \prod_{i=1}^D p(x_i \mid \mathbf{x}_{<\mathbf{i}})$$

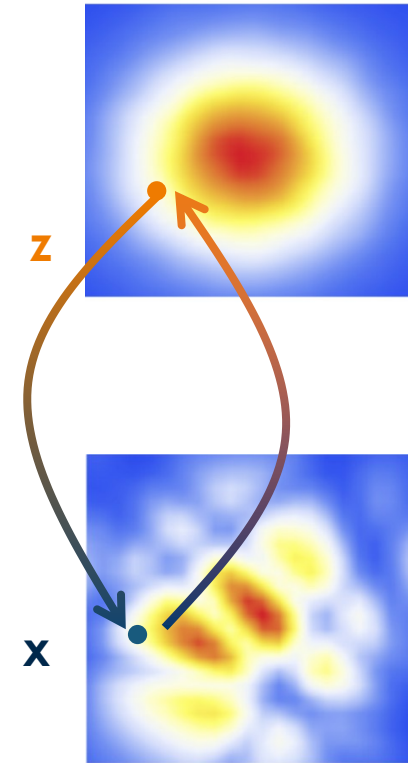
Lessons from other model families

- Variational autoencoders can learn feature representations but exact likelihood is untractable
- Autoregressive models provide tractable likelihoods but no direct mechanism for learning features

Question: can we design a latent variable model that can learn feature representations and has tractable likelihoods?

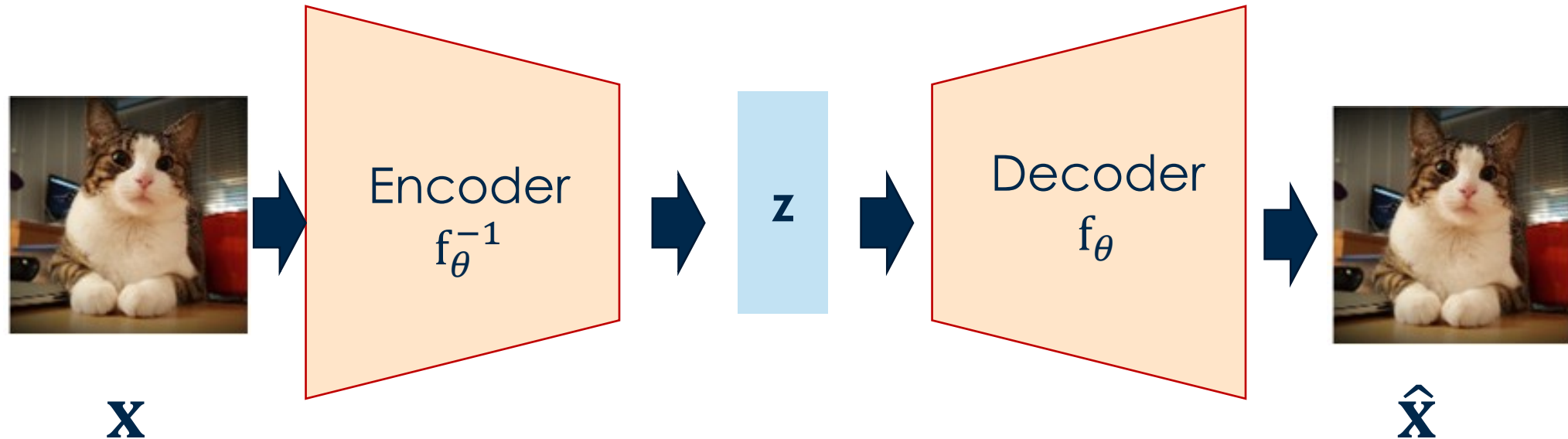
Simple Prior to Complex Data Distributions

- Desirable properties of any model distribution $p_{\theta}(x)$:
 - › Easy-to-evaluate, closed form density (useful for training)
 - › Easy-to-sample (useful for generation)
- Unfortunately, data distributions are more complex (multi-modal)



Key idea: Map simple distributions (easy to sample and evaluate densities) to complex distributions through an **invertible transformation**.

Normalizing flows: core concept



Make $\mathbf{x} = f_{\theta}(\mathbf{z})$ a deterministic and invertible function of $\mathbf{z}$, so for any $\mathbf{x}$ there is a unique corresponding $\mathbf{z}$

Note: $\mathbf{x}$ and $\mathbf{z}$ are both continuous and have the same dimension

Training normalizing flows (NF)

- NFs model a complex data distribution $p_{\text{data}}(\mathbf{x})$ by transforming a simple prior $p_{\text{prior}}(\mathbf{z}) = \mathcal{N}(0, I)$ via an invertible mapping $f_{\phi}: \mathbb{R}^D \rightarrow \mathbb{R}^D$, with $\mathbf{x} = f_{\phi}(\mathbf{z})$ and $\mathbf{z} \sim p_{\text{prior}}$
- Parameters ϕ are learned by maximizing the likelihood over data:

$$\mathcal{L}_{\text{NF}}(\phi) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log p_{\phi}(\mathbf{x})]$$

Key idea: Make the learning objective easy to learn by exploiting the properties of the invertible mapping

Training normalizing flows (NF)

- Given an invertible transformation $f(\cdot)$, we know that the density of $x = f(z)$ where $z \sim p_{\text{prior}}$ is given by this formula:

$$p(x) = p_{\text{prior}}(z) \left| \det \frac{\partial f^{-1}(x)}{\partial x} \right|, \quad \text{where } z = f^{-1}(x)$$

$$J = \frac{\partial f(\mathbf{z})}{\partial \mathbf{z}} = \begin{pmatrix} \frac{\partial f_1}{\partial z_1} & \dots & \frac{\partial f_1}{\partial z_d} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_d}{\partial z_1} & \dots & \frac{\partial f_d}{\partial z_d} \end{pmatrix}$$

- Each entry J_{ij} measures how much the output dimension i changes when input dimension j changes.
- The determinant $|\det J|$ measures how much the transformation **locally scales volumes**.

Training normalizing flows (NF)

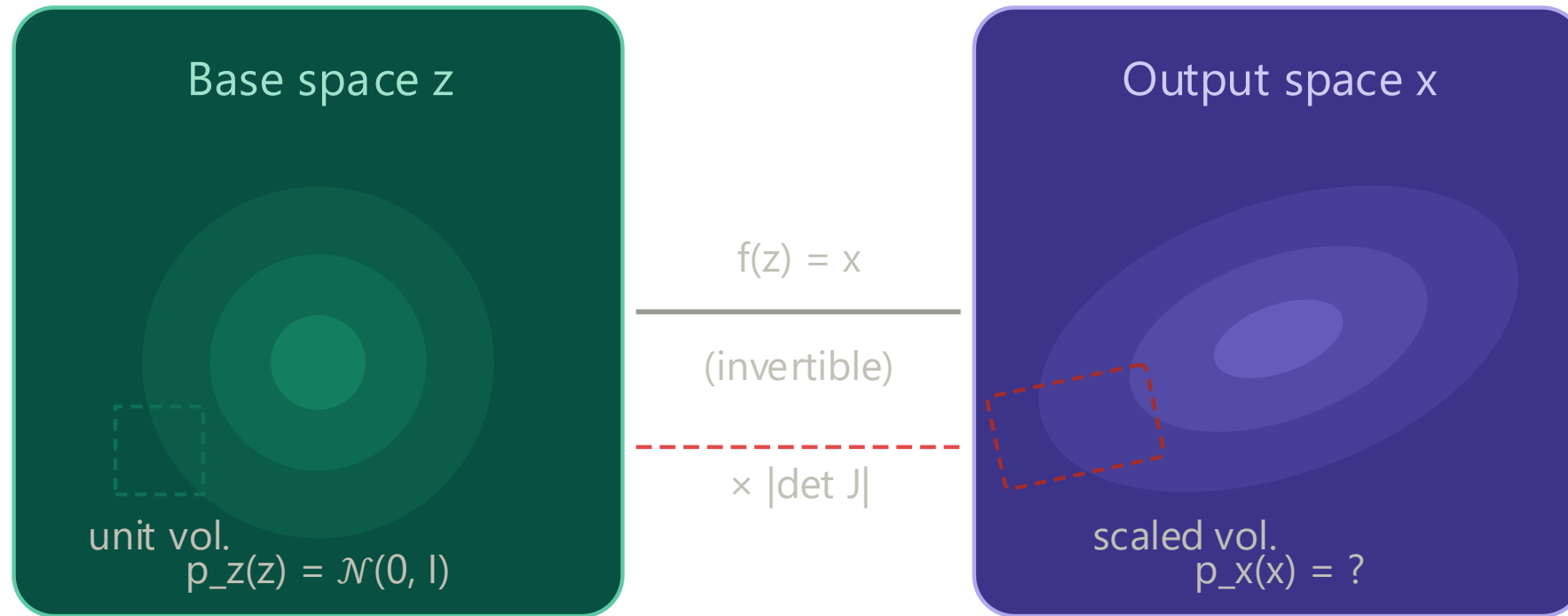
- Given an invertible transformation $f(\cdot)$, we know that the density of $x = f(z)$ where $z \sim p_{\text{prior}}$ is given by this formula:

$$p(x) = p_{\text{prior}}(z) \left| \det \frac{\partial f^{-1}(x)}{\partial x} \right|, \quad \text{where } z = f^{-1}(x)$$

- Applying this change of variable, we obtain the training objective:

$$\underbrace{\log p(\mathbf{x})}_{\text{Log-density of output}} = \underbrace{\log p_{\text{prior}}(\mathbf{z})}_{\text{Log-density of input}} + \underbrace{\log \left| \det \frac{\partial f_{\theta}^{-1}(\mathbf{x})}{\partial \mathbf{x}} \right|}_{\text{Volume change correction}}$$

Loss: geometric interpretation



Learning and inference

- Learning via **maximum likelihood** over the dataset D
- **Exact likelihood evaluation** via inverse transformation $\hat{\mathbf{x}} \rightarrow \mathbf{z}$ and change-of-variables formula
- **Sampling** via forward transformation $\hat{\mathbf{z}} \rightarrow \mathbf{x}$
- **Latent representations** inferred via inverse transformation $\mathbf{z} = \mathbf{f}^{-1}(\mathbf{x})$
 - no inference network required!

Desiderata for flow models

- Simple prior $p_z(\mathbf{z})$
 - Efficient sampling
 - Tractable likelihood
- Invertible transformations with tractable evaluation
 - Likelihood evaluation requires efficient evaluation of $\mathbf{x} \rightarrow \mathbf{z}$ mapping
 - Sampling requires efficient evaluation of $\mathbf{z} \rightarrow \mathbf{x}$ mapping
- Computing likelihoods also requires the evaluation of determinants of $n \times n$ Jacobian matrices, where n is the data dimensionality
 - Computing $\det J$ is expensive
 - For a d -dimensional transformation, J is a $d \times d$ matrix. Computing its determinant naively costs $O(d^3)$ — completely intractable for images ($d = 256 \times 256 = 65536$)

Which transformation?

The solution: Design transformation wisely

- If f is designed so that J is lower-triangular, the determinant is just the product of the diagonal elements — the complexity is $O(d)$.

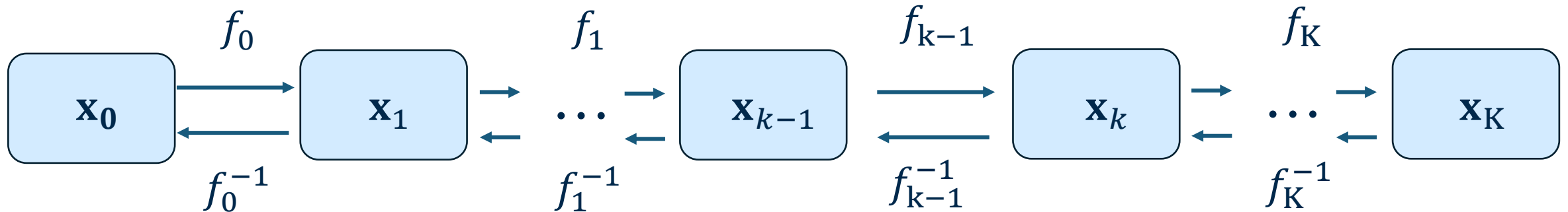
$$\det J = \prod_{i=1}^d J_{ii}$$

Chaining transformation

A single complex invertible network can be expensive due to its Jacobian determinant. Conversely, simple transforms (e.g., linear) are efficient but lack expressivity.

Chaining transformation

A single complex invertible network can be expensive due to its Jacobian determinant. Conversely, simple transforms (e.g., linear) are efficient but lack expressivity.



Solution: chaining K simple transformations can represent **arbitrarily complex distributions**, while each individual transformation remains simple enough to be inverted efficiently and have a tractable Jacobian determinant

Chaining transformation

- Since we are chaining K simple transformations instead of computing a single complex one:

$$f_{\phi}(\mathbf{z}_0) = f_{K-1} \circ f_{K-2} \circ \cdots \circ f_0(\mathbf{z}_0)$$

$$f_{\phi}(\mathbf{z}_0) = f_{K-1} \left(f_{K-2} \left(\cdots f_1 \left(f_0(\mathbf{z}_0) \right) \cdots \right) \right)$$

- Each intermediate $\mathbf{z}_k$ is called a **latent representation** at layer k.

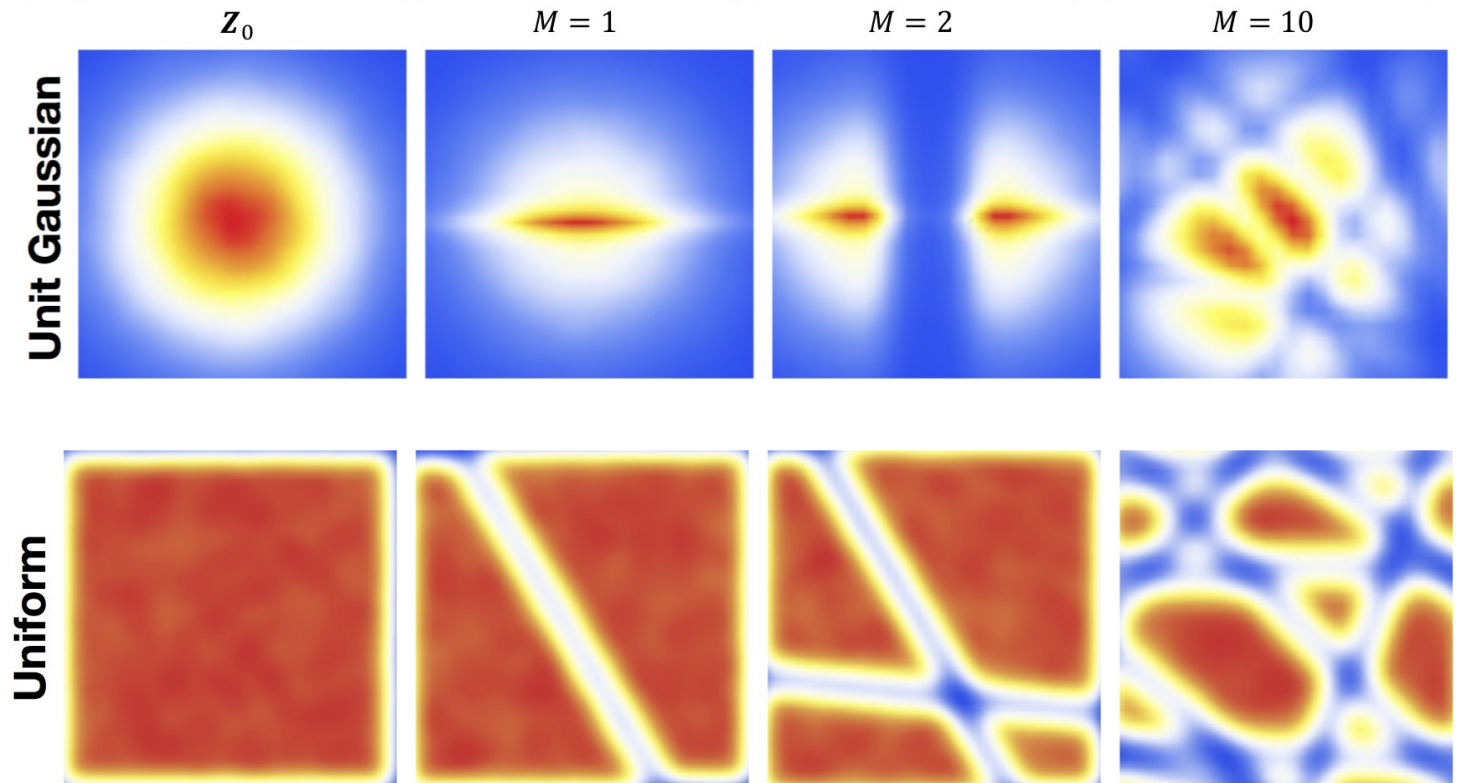
$$p_{\phi}(\mathbf{x}) = p_{\text{prior}}(\mathbf{x}_0) \prod_{k=0}^{K-1} \left| \det \frac{\partial f_k}{\partial \mathbf{z}_k} \right|^{-1} \quad \Rightarrow \quad \log p_x(\mathbf{x}) = \log p_z(\mathbf{z}_0) - \sum_{k=0}^{K-1} \log \left| \det \frac{\partial f_k}{\partial \mathbf{z}_k} \right|$$

Example: planar flows

Base distribution: **Gaussian**

$$\mathbf{x} = \mathbf{f}_\theta(\mathbf{z}) = \mathbf{z} + \mathbf{u}h(\mathbf{w}^T\mathbf{z} + b)$$

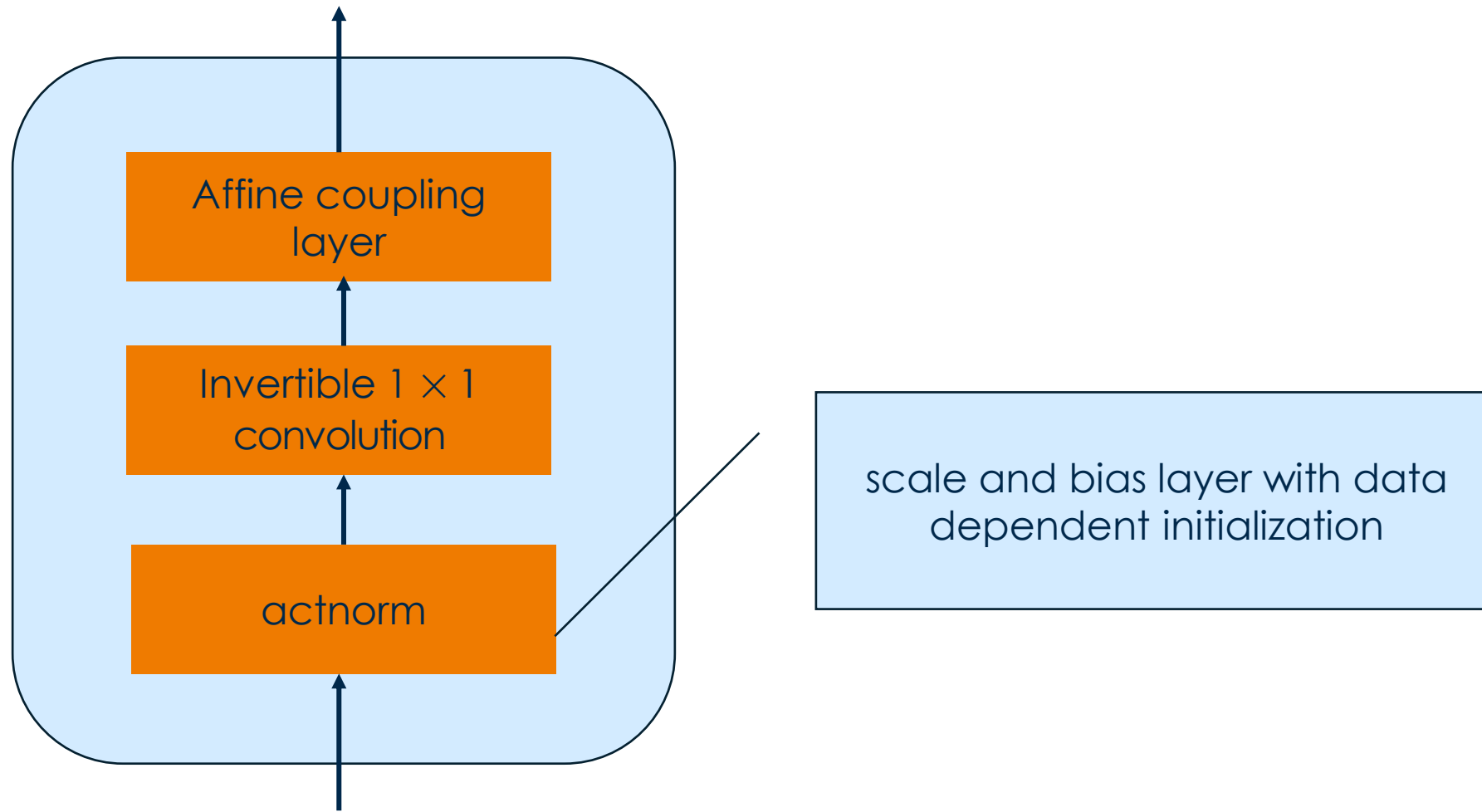
Base distribution: **Uniform**



10 planar transformations can transform simple distributions into a more complex one

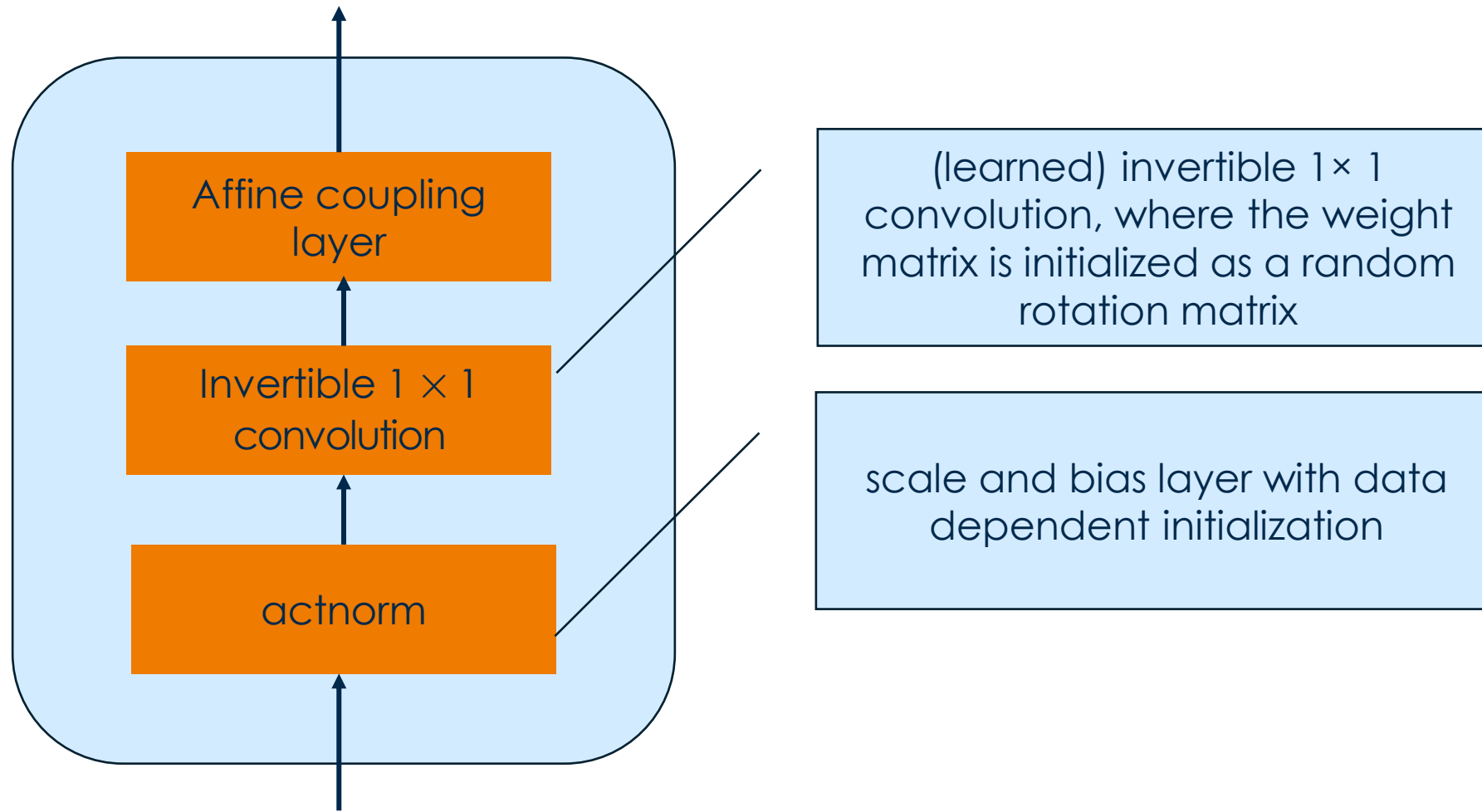
Rezende, Danilo, and Shakir Mohamed. "Variational inference with normalizing flows." *ICML*, 2015.

GLOW



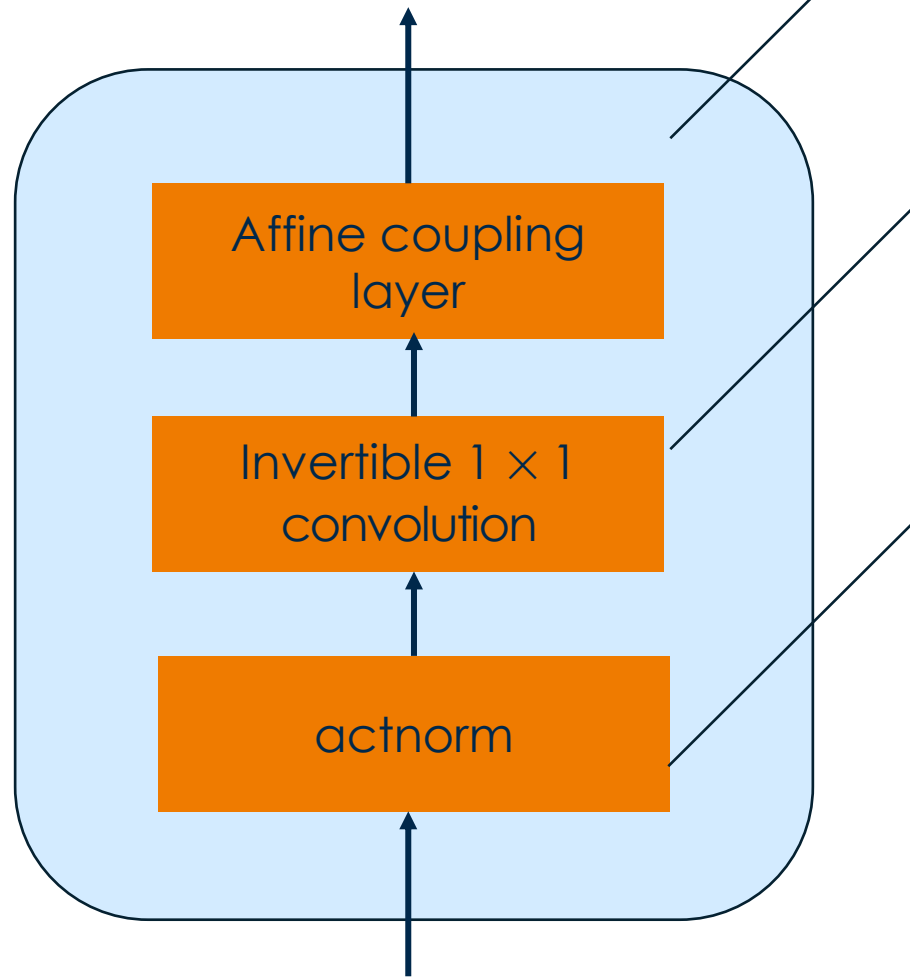
Kingma, Durk P., and Prafulla Dhariwal. "Glow: Generative flow with invertible 1x1 convolutions." *NeurIPS* 31 (2018).

GLOW



Kingma, Durk P., and Prafulla Dhariwal. "Glow: Generative flow with invertible 1x1 convolutions." *NeurIPS* 31 (2018).

GLOW

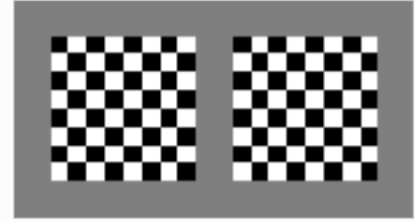


Variables are split based on mask
- half is transformed using affine transformation

(learned) invertible 1×1 convolution, where the weight matrix is initialized as a random rotation matrix

scale and bias layer with data dependent initialization

Checkerboard mask



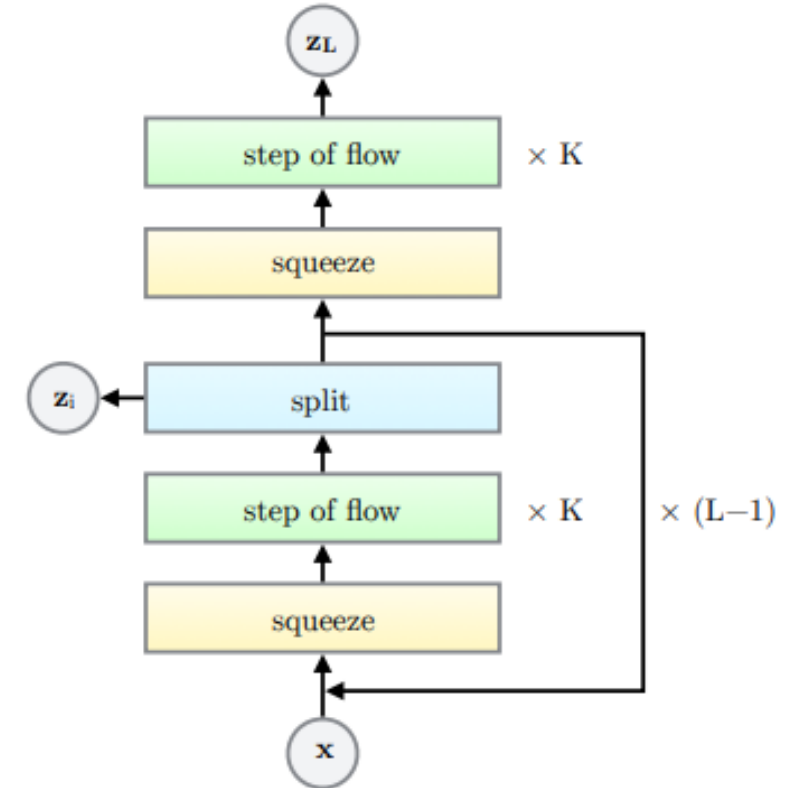
Channel mask



Kingma, Durk P., and Prafulla Dhariwal. "Glow: Generative flow with invertible 1×1 convolutions." *NeurIPS* 31 (2018).

GLOW

- One disadvantage of normalizing flows is that they operate on the exact same dimensions as the input
- Deep normalizing flows on images commonly apply a multi-scale architecture
- After the first K flow transformations, we split off half of the latent dimensions and directly evaluate them on the prior, while the other are run through the remaining steps, up to L .



Kingma, Durk P., and Prafulla Dhariwal. "Glow: Generative flow with invertible 1x1 convolutions." *NeurIPS* 31 (2018).

From discrete to continuous flows

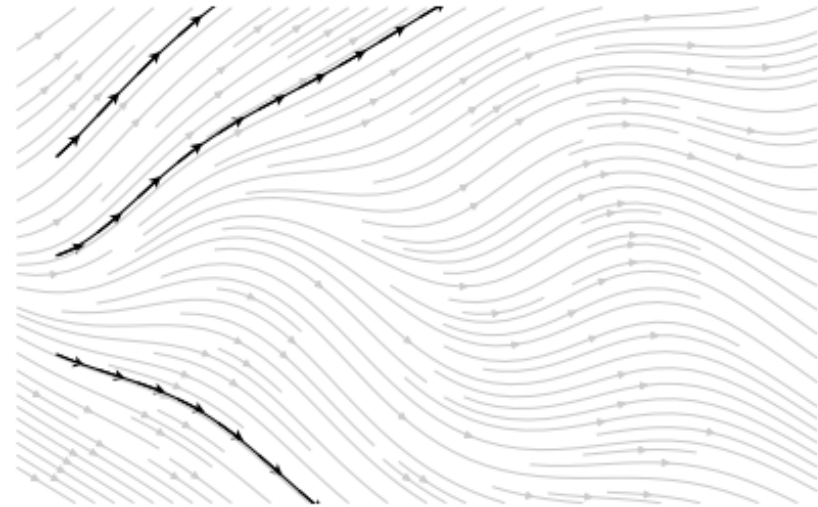
Residual flows: each transformation moves points along the directions suggested by this velocity, gradually morphing the simple prior distribution into the complex target distribution.

$$\mathbf{x}_{k+1} = f_k(\mathbf{x}_k) := \mathbf{x}_k + v_k(\mathbf{x}_k, k)$$



Continuous flows: residual flows can be seen as a discretization of an ODE. The neural network estimates the velocity field and the terminal distribution at $t = T$ matches the target distribution $p_{\text{data}}(\cdot)$.

$$\frac{d\mathbf{x}(t)}{dt} = \mathbf{v}_{\phi}(\mathbf{x}(t), t)$$



<https://arxiv.org/pdf/2510.21890>, pp. 136

Major generative model families

Autoregressive Models

GPT, PixelCNN, WaveNet

- Computes exact likelihood
- + Multi-modal, flexible model
- Slow inference (token-by-token)
- Depends on tokenization

Variational Autoencoders

VAE, β -VAE, VQVAE

- Explicit latent space modelling
- May serve as building blocks for other models
- + Interpretability of the feature space
- + Fast inference
- + Effective representation learning
- Blurry images, low realism

GANs

GAN, StyleGAN, BigGAN

- Implicit distribution
- + Fast inference (single step)
- + Photo-realism
- Unstable training

Normalizing Flows

Glow, RealNVP, MAF

- + Exact likelihood computation
- + Explicit latent space modelling
- Requires invertible function
- Computational expensive
- Good (not exceptional) quality

Diffusion Models

DDPM, Stable Diffusion, DALL-E

- State of the art
- Denoising in latent space
- + High quality image synthesis
- + Flexible and highly controllable synthesis
- Slow inference
- Requires massive pretraining

Energy-Based Models

EBM, NCSN, ScoreNet

Flexible density